FSR Scraping & Metadata Extraction – Proof of Concept Documentation

1. Introduction

This Proof of Concept (POC) was initiated to automate the extraction, normalization, and enrichment of metadata from Field Service Reports (FSRs). The goal is to streamline the process of converting unstructured PDF documents into structured datasets that can be used for analytics, reporting, and downstream integration with enterprise systems.

The effort was divided into three major stages:

1. **Extraction** – Parsing PDF files to capture key fields and metadata.
2. **Normalization** – Using AI/LLM models to standardize the extracted data into a consistent schema.
3. **Enrichment** – Integrating normalized data with enterprise reference tables (IBAT Equipment Master and Event Vision SOT) to fill gaps and enhance quality.

2. Stage 1 – PDF Extraction

Objective

Extract raw metadata from FSR PDF files in a flexible manner, capturing both titles and key-value fields.

Implementation

- **Tooling:** Implemented in Python using the pdfplumber library.
- **Process:**
    - Iterates through all PDFs in a designated folder.
    - Extracts text from the first page of each PDF.
    - Captures the **title** (first 2–3 lines before any colon-separated fields).
    - Identifies and stores **key:value pairs**, supporting multi-line values.
    - Handles duplicate keys by concatenating values.
- **Output:** JSON file containing a list of extracted records, each with:
    - PDF identifier (path/name)
    - Title
    - Key-value metadata fields

Key Features

- Robust error handling (logs errors but continues processing).
- Configurable limits (process all PDFs or restrict to a subset).
- Flexible parsing logic to handle variations in PDF formatting.

3. Stage 2 – Data Normalization

Objective

Transform the raw extracted metadata into a structured, standardized schema suitable for analysis, with refined handling of EV/EVP/EV Equipment Event IDs.

Implementation

- **Tooling:** Python + litellm library for LLM integration.
- **Process:**
    - Loads extracted JSON records.
    - Processes records in batches (default: 50 PDFs per batch).
    - Sends batch data to an LLM with a detailed prompt specifying:
        - Required fields (e.g., ESN, Equipment Sys ID, Event Type, Report Dates).
        - Mapping rules for project IDs:
            - Oracle Project IDs → OFS Event ID (excluding EV/EVP prefixes).
            - EV- → EV Equipment Event ID.
            - EVP- → EV Project ID.
            - FSP- → FSP Project ID.
            - XXX prefixes (A-, C-, etc.) → xxx project id.
        - Controlled vocabulary for Event Type.
        - Splitting logic for multiple ESN/Equipment Sys IDs.
    - Receives normalized JSON output from the LLM.
    - Converts results into both JSON and Excel formats.

Output

- Normalized JSON with consistent schema.
- Excel file for easy review and downstream processing.

Key Features

- Intelligent handling of ambiguous fields (infers values from context).
- Strict enforcement of allowed values for Event Type.
- Splitting of complex records into multiple rows while preserving data integrity.
- Updated mapping logic for EV/EVP/EV Equipment Event IDs.

4. Stage 3 – Data Enrichment

Objective

Enhance normalized FSR metadata by joining with enterprise reference data (IBAT Equipment Master and Event Vision SOT).

Implementation

- **Tooling: Python + PySpark.**
- **Process:**
    - Loads normalized Excel into Pandas, then converts to Spark DataFrame.
    - Normalizes column names (ESN, Equipment Sys ID, etc.).
    - Cleans and strips prefixes (EV-, EVP-, FSP-) for join keys.
    - Joins FSR dataset with IBAT Equipment Master table on equipment_sys_id or ESN.
    - Fills missing values for ESN, Equipment Type, and Equipment Sub-Class using IBAT data.
    - Joins enriched dataset with Event Vision SOT table using EV, EVP, OFS, and FSP keys.
    - Fills missing values for project IDs, event types, and outage dates from Event Vision SOT.
    - Restores original column order for consistency.
    - Exports enriched dataset as a plain text file (CSV-style).

Output

- Final enriched dataset in text format, ready for downstream analytics.

Key Features

- Ensures completeness by filling gaps from IBAT and Event Vision SOT reference data.
- Maintains row order and original schema for traceability.
- Enhanced join logic with multiple keys (EV, EVP, OFS, FSP).
- Prefix restoration for IDs when filling missing values.

5. Achievements

- Automated extraction of metadata from hundreds of FSR PDFs.
- Standardized schema creation using AI-assisted normalization.
- Integration with enterprise equipment master data for enrichment.
- Multi-format outputs (JSON, Excel, text) for flexibility.
- Modular pipeline design, allowing independent improvements at each stage.

6. Conclusion

The POC successfully demonstrates the feasibility of automating FSR metadata extraction and enrichment. By combining PDF parsing, AI-driven normalization, and Spark-based enrichment, the pipeline transforms unstructured reports into structured, high-quality datasets. This lays the foundation for improved analytics, reporting, and operational efficiency in handling FSR data.

Appendix – Code Implementations

This appendix contains the full code implementations developed as part of the FSR Scraping & Metadata Extraction Proof of Concept. The snippets are presented exactly as implemented, without modifications.

Appendix A – PDF Extraction

```
%pip install pdfplumber


import os
import pdfplumber
import json


# ----------------------------
# Config
# ----------------------------
MAIN_PDF_FOLDER = "/Volumes/vgpd/fsr_std_views/fsr_std_vol/FSR_After_2016/FSR_Databricks/"
OUTPUT_JSON = "/Volumes/vgpd/fsr_std_views/fsr_std_vol/data/n_pdfs_extracted_title_2016.json"
MAX_PDFS = None   # <-- no limit, process all PDFs


# ----------------------------
# Flexible Extraction with Multi-line Support + Title
# ----------------------------
def extract_fields_from_pdf(pdf_path):
  fields = {"PDF Name / path / identifier": pdf_path}


  try:
    with pdfplumber.open(pdf_path) as pdf:
      first_page = pdf.pages[0]
      text = first_page.extract_text()
```

```
        if text:
            lines = text.splitlines()
            current_key = None

            # --- Capture title (first 2–3 lines before any colon fields) ---
            title_lines = []
            for line in lines:
                if ":" in line:
                    break
                title_lines.append(line.strip())
            if title_lines:
                fields["Title"] = " ".join(title_lines)

            # --- Process key:value fields ---
            for line in lines:
                if ":" in line:
                    parts = line.split(":", 1)
                    key = parts[0].strip()
                    value = parts[1].strip()

                    current_key = key
                    if key in fields:
                        fields[key] = f"{fields[key]} | {value}"
                    else:
                        fields[key] = value
                else:
                    if current_key:
                        fields[current_key] += " " + line.strip()

    except Exception as e:
```

```
        print(f"Error extracting {pdf_path}: {e}")

    return fields

# ----------------------------
# Main Pipeline
# ----------------------------
def main():
    results = []
    count = 0

    for root, dirs, files in os.walk(MAIN_PDF_FOLDER):
        for filename in files:
            if filename.lower().endswith(".pdf"):
                if MAX_PDFS and count >= MAX_PDFS:
                    print(f"Reached limit of {MAX_PDFS} PDFs. Stopping.")
                    break

                pdf_path = os.path.join(root, filename)
                print(f"Processing: {pdf_path}")
                data = extract_fields_from_pdf(pdf_path)
                results.append(data)
                count += 1
        if MAX_PDFS and count >= MAX_PDFS:
            break

    output = {"FSR_data": results}

    with open(OUTPUT_JSON, "w", encoding="utf-8") as f:
        json.dump(output, f, indent=4, ensure_ascii=False)
```

```
    print(f"Extraction complete. Processed {count} PDFs. Results saved to {OUTPUT_JSON}")

if __name__ == "__main__":
    main()
```

Appendix B – Data Normalization

```
import json
import pandas as pd
import os
import litellm

# ----------------------------
# Config
# ----------------------------
INPUT_JSON = "n_pdfs_extracted_title_2016_Vinayaka.json"   # extracted fields JSON
OUTPUT_JSON = "llm_filtered_title_2016_Vinayaka_EV_EVP.json"      # normalized output
OUTPUT_EXCEL = "llm_filtered_title_2016_Vinayaka_EV_EVP.xlsx"      # final Excel file
BATCH_SIZE = 50                  # number of PDFs per LLM call (tune as needed)

# ----------------------------
# LLM call functions
# ----------------------------

def call_llm(messages):
    os.environ['SSL_VERIFY'] = 'False'
    MODELS= ['azure-gpt-4o', 'azure-gpt-5o',  'vertex-ai-gemini-2.5-flash', 'azure-gpt-4-1']

    API_BASE="https://sandbox-llmproxy.research.gevernova.net"
    API_KEY= "********"
```

```
    os.environ['SSL_VERIFY'] = 'False'

    response =  litellm.completion(
            model= f'litellm_proxy/{MODELS[1]}',
            messages=messages,
            api_base=API_BASE,
            api_key=API_KEY,
            # temperature=0.1, #can't use
            # max_token= 128000 azure-gpt-4o
        )
    return response


def invoke_client_llm_outline(prompt: str):

    system_prompt = """
    You are an expert in structuring technical data.
    Your task is to process the provided input json and output a structured/normalized JSON format according to user instructions.
    Focus on clarity, completeness, and following the JSON schema provided. Do not include any internal reasoning or system details in the output.
    Ensure all responses are fact-based, and safe.

    Follow responsible AI principles without over-restricting harmless tasks.
    """

    messages = [{'role':'system', 'content': system_prompt},
            {'role': 'user', 'content': prompt}]

    response = call_llm(messages)
    # print(response)
```

```
    response = response.choices[0].message.content
    return response



# ----------------------------
# Main Pipeline
# ----------------------------
def main():
    # Load extracted JSON
    with open(INPUT_JSON, "r", encoding="utf-8") as f:
        data = json.load(f)

    # If wrapped in {"FSR_data": [...]}, unwrap
    records = data.get("FSR_data", data)

    results = []

    # Process in batches
    for i in range(0, len(records), BATCH_SIZE):
        batch = records[i:i+BATCH_SIZE]

        # For extracting pdfs after 2016 alone. Splitting rows based on multiple entries into different rows.
        prompt = f"Here is a JSON list of PDF fields:\n{json.dumps(batch, indent=2)}\n\n"
        prompt += """Your task is to process JSON input and output a normalized JSON format
        with the following fields only:

        ESN, Equipment Sys ID, Equipment Type, Equipment Class / Code, Event Type,
        EV Project ID, EV Equipment Event ID, OFS Event ID, FSP project ID, xxx project id, PDF Name / path / identifier,
        FSR Number (#), Report Issued Date, Outage Start Date, Outage End Date.
```

If the PDF field contains an Oracle Project Id, map it to OFS Event ID only. But do not include field values that start with "EV" and "EVP" under OFS Event ID.

Map the PDF field containing "EV-" to EV Equipment Event ID only.

Map the PDF field containing "EVP-" to EV Project ID only.

If the PDF field contains a Field Service Project Id or "FSP-", map it to FSP project ID only.

If the PDF filed contains a project id that starts with "XXX" (i.e. A-, C-, etc.), map it to xxx project id only.

Do not miss any information that is present in the input and try to be as much accurate as possible in retrieving the values for the above fields.

When extracting data, if a record contains multiple distinct values across ESN and Equipment Sys ID fields, split the record into separate rows by pairing values positionally (first with first, second with second, etc.), while duplicating all other field values unchanged (except for Equipment Type and Equipment Class / Code). Set the Equipment Type and Equipment Class / Code field values to empty strings in the split rows. Do not split or omit any parts for other field values even if multiple distinct values are present.

If no exact match is found for a field, see if you can infer it from similar labels or context.

If no relevant information is found, output it as an empty string.

Consider as many records as provided in the input batch. Do not omit any records.

Do not include any reasoning or commentary, only valid JSON output.

For the field Event Type, only use values from the following allowed list:

Training Cost Accumulation

Unusual

Training Open Enrollment - Costs

TX Repairs

Major Inspection (Field Rewind)

Major Inspection (MI)

Tooling(GE)

C Inspection

null

Training On Site Training

A Inspection

Services Warranty

Borescope Inspection (BI)

Major Inspection (Robotic)

Performance Testing

Upgrade - PMO Billing only

Digital

Non CSA-MMP Billing

Hot Gas Path Inspection (HGPI)

Initial Spares

Combustion Inspection (CI)

Training Open Enrollment - Billing

Stand Alone Small Upgrade

Large Call-Out

On Site Services

Call-Out

TX Parts

B Inspection

Training Simulation

Post COD New Unit Warranty

Major Inspection (Rotor Out)

Stand Alone Large Upgrade

OP Spares

Remote Diagnostics

Minor Inspection

Onsite Services SP

Major Inspection (Stator Rewind)

"""

Call LLM

```
        print(f"Processing batch {i//BATCH_SIZE + 1}...")
        response_text = invoke_client_llm_outline(prompt)

        try:
            batch_result = json.loads(response_text)
            # If wrapped in dict, unwrap
            if isinstance(batch_result, dict) and "FSR_data" in batch_result:
                batch_result = batch_result["FSR_data"]
            results.extend(batch_result)
        except Exception as e:
            print(f"Error parsing LLM response for batch {i//BATCH_SIZE + 1}: {e}")
            print("Raw response:", response_text)

    # Save combined results to JSON
    output = {"FSR_data": results}
    with open(OUTPUT_JSON, "w", encoding="utf-8") as f:
        json.dump(output, f, indent=4, ensure_ascii=False)

    print(f"Normalization complete. Results saved to {OUTPUT_JSON}")

    # ----------------------------
    # Export to Excel
    # ----------------------------
    df = pd.DataFrame(results)
    df.to_excel(OUTPUT_EXCEL, index=False)
    print(f"Excel export complete. Results saved to {OUTPUT_EXCEL}")

if __name__ == "__main__":
    main()
```

Appendix C – Data Enrichment

```
#%pip install openpyxl


import pandas as pd
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
   col, when, monotonically_increasing_id, trim, concat, lit, upper, regexp_replace
)
from pyspark.sql.types import StringType


# Initialize Spark
spark = SparkSession.builder.getOrCreate()


# Paths
fsr_xl_path = "/Volumes/vgpd/fsr_std_views/fsr_std_vol/data/llm_filtered_title_2016_Vinayaka_EV_EVP.xlsx"
text_output_path = "/Volumes/vgpd/fsr_std_views/fsr_std_vol/data/fsr_enriched_2016_Vinayaka_combined.txt"


# Step 1: Load Excel into pandas and normalize column names for internal processing
fsr_pd = pd.read_excel(fsr_xl_path)
fsr_pd = fsr_pd.rename(columns={
   "ESN": "esn",
   "Equipment Sys ID": "equipment_sys_id",
   "Equipment Type": "equipment_type",
   "Equipment Class / Code": "equipment_sub_class",
   "EV Project ID": "ev_project_id",
   "EV Equipment Event ID": "ev_equipment_event_id",
   "Event Type": "ev_event_type",
   "OFS Event ID": "ofs_event_id",
```

```
    "FSP project ID": "fsp_project_id",
    "Outage Start Date": "p6_outage_start_date",
    "Outage End Date": "p6_outage_end_date"
})

# Preserve original Excel column names for final export
original_excel_cols = list(pd.read_excel(fsr_xl_path).columns)

# Step 2: Convert to Spark DataFrame and add row index
fsr_df = spark.createDataFrame(fsr_pd)
fsr_df = fsr_df.select([col(c).cast(StringType()).alias(c) for c in fsr_df.columns])

# Trim blanks and normalize IDs
fsr_df = (
    fsr_df
    .withColumn("ev_project_id", trim(col("ev_project_id")))
    .withColumn("ev_equipment_event_id", trim(col("ev_equipment_event_id")))
    .withColumn("ofs_event_id", trim(col("ofs_event_id")))
    .withColumn("fsp_project_id", trim(col("fsp_project_id")))
    # strip FSP- prefix for join
     .withColumn("fsp_project_id_stripped",  regexp_replace(col("fsp_project_id"),  "FSP-", ""))
    # strip EV- / EVP- prefixes for join
    .withColumn("ev_project_id_stripped", regexp_replace(col("ev_project_id"), "EVP-", ""))
                                            .withColumn("ev_equipment_event_id_stripped", regexp_replace(col("ev_equipment_event_id"), "EV-", ""))
    .withColumn("row_index", monotonically_increasing_id())
)

# Step 3: Load IBAT Equipment Master and rename overlapping columns
ibat_equipment = spark.read.table("vgpd.prm_std_views.ibat_equipment_mst") \
```

```
    .withColumnRenamed("equipment_sys_id", "ibat_equipment_sys_id") \
    .withColumnRenamed("equipment_type", "ibat_equipment_type") \
    .withColumnRenamed("equipment_sub_class", "ibat_equipment_sub_class") \
    .withColumnRenamed("equip_serial_number", "ibat_equip_serial_number")


ibat_equipment = ibat_equipment.select([col(c).cast(StringType()).alias(c) for c in ibat_equipment.columns])
ibat_equipment = (
    ibat_equipment
    .withColumn("ibat_equipment_sys_id", upper(trim(col("ibat_equipment_sys_id"))))
                                                .withColumn("ibat_equip_serial_number", upper(trim(col("ibat_equip_serial_number"))))
)

# Step 4: Join with IBAT
enriched = (
    fsr_df.join(
        ibat_equipment,
        (fsr_df["equipment_sys_id"] == ibat_equipment["ibat_equipment_sys_id"]) |
        (fsr_df["esn"] == ibat_equipment["ibat_equip_serial_number"]),
        "left"
    )
)

# Step 5: Fill missing values from IBAT
enriched = (
    enriched
    .withColumn("esn",
        when((col("esn").isNull()) | (col("esn") == ""), col("ibat_equip_serial_number"))
        .otherwise(col("esn")))
    .withColumn("equipment_sys_id",
```

```
            when((col("equipment_sys_id").isNull()) | (col("equipment_sys_id") == ""), col("ibat_equipment_sys_id"))
      .otherwise(col("equipment_sys_id")))
   .withColumn("equipment_type",
               when((col("equipment_type").isNull()) | (col("equipment_type") == ""), col("ibat_equipment_type"))
      .otherwise(col("equipment_type")))
   .withColumn("equipment_sub_class",
       when((col("equipment_sub_class").isNull()) | (col("equipment_sub_class") == ""), col("ibat_equipment_sub_class"))
      .otherwise(col("equipment_sub_class")))
)

# Step 6: Load Event Vision SOT table and rename overlapping columns
event_sot = spark.read.table("vgpd.fsr_std_views.eventmgmt_event_vision_sot") \
   .withColumnRenamed("ev_project_id", "sot_ev_project_id") \
   .withColumnRenamed("ev_equipment_event_id", "sot_ev_equipment_event_id") \
   .withColumnRenamed("ev_gtm_id", "sot_ev_gtm_id") \
   .withColumnRenamed("fsp_project_id", "sot_fsp_project_id") \
   .withColumnRenamed("ev_event_type", "sot_ev_event_type") \
   .withColumnRenamed("p6_outage_start_date", "sot_outage_start_date") \
   .withColumnRenamed("p6_outage_end_date", "sot_outage_end_date")

event_sot = event_sot.select([col(c).cast(StringType()).alias(c) for c in event_sot.columns])

# Step 7: Join with Event Vision SOT using EV, EVP, OFS, and FSP keys
enriched = (
   enriched.join(
      event_sot,
     (enriched["ev_project_id_stripped"] == event_sot["sot_ev_project_id"]) |  # EVP match
```

```
                                    (enriched["ev_equipment_event_id_stripped"] ==
event_sot["sot_ev_equipment_event_id"]) | # EV match
      (enriched["ofs_event_id"] == event_sot["sot_ev_gtm_id"]) |
      (enriched["fsp_project_id_stripped"] == event_sot["sot_fsp_project_id"]),
      "left"
   )
)

# Step 8: Fill missing values from Event Vision SOT
enriched = (
   enriched
   .withColumn("ev_project_id",
      when((col("ev_project_id").isNull()) | (col("ev_project_id") == ""),
         concat(lit("EVP-"), col("sot_ev_project_id")))
      .otherwise(col("ev_project_id")))
   .withColumn("ev_equipment_event_id",
      when((col("ev_equipment_event_id").isNull()) | (col("ev_equipment_event_id") == ""),
         concat(lit("EV-"), col("sot_ev_equipment_event_id")))
      .otherwise(col("ev_equipment_event_id")))
   .withColumn("ofs_event_id",
      when((col("ofs_event_id").isNull()) | (col("ofs_event_id") == ""), col("sot_ev_gtm_id"))
      .otherwise(col("ofs_event_id")))
   .withColumn("fsp_project_id",
      when((col("fsp_project_id").isNull()) | (col("fsp_project_id") == ""),
         concat(lit("FSP-"), col("sot_fsp_project_id")))
      .otherwise(col("fsp_project_id")))
   .withColumn("ev_event_type",
                  when((col("ev_event_type").isNull())  |  (col("ev_event_type")  ==  ""),
col("sot_ev_event_type"))
      .otherwise(col("ev_event_type")))
   .withColumn("p6_outage_start_date",
```

```
        when((col("p6_outage_start_date").isNull()) | (col("p6_outage_start_date") == ""), col("sot_outage_start_date"))
      .otherwise(col("p6_outage_start_date")))
   .withColumn("p6_outage_end_date",
         when((col("p6_outage_end_date").isNull()) | (col("p6_outage_end_date") == ""), col("sot_outage_end_date"))
      .otherwise(col("p6_outage_end_date")))
)

# Step 9: Select only original Excel columns + row_index
enriched = enriched.select([col(c) for c in fsr_pd.columns] + [col("row_index")])

# Step 10: Restore row order and drop row_index
enriched = enriched.orderBy("row_index").drop("row_index")

# Step 11: Convert to Pandas and save as plain text (CSV-style) with original Excel headers
enriched_pd = enriched.toPandas()

with open(text_output_path, "w", encoding="utf-8") as f:
   f.write(",".join(final_cols) + "\n")
   for row in enriched_pd.itertuples(index=False):
      f.write(",".join([str(x) if x is not None else "" for x in row]) + "\n")

print(f"Enrichment complete. Final text file saved to {text_output_path}")
```